CivicOS Engineering Playbook

Building Software That Lasts

Version: 1.0

Status: Draft

Chapter 1

Introduction

Purpose

The CivicOS Engineering Playbook defines the engineering standards, development practices, architectural principles, and operational guidelines used to build and maintain CivicOS.

Unlike the Vision Document, Technical Architecture Document, Experience Architecture Document, and Product Roadmap, this document is intended primarily for engineers.

Its purpose is to ensure that every contributor, regardless of experience or location, builds software consistently.

Whether a contributor is fixing a bug, implementing a new service, designing an API, or deploying infrastructure, this playbook provides the shared engineering language that keeps CivicOS maintainable over the long term.

Objectives

The Engineering Playbook exists to achieve the following objectives:

- Establish a consistent engineering culture.

- Define technical standards across the project.
- Reduce onboarding time for contributors.
- Improve software quality.
- Minimize architectural drift.
- Encourage sustainable development.
- Support open-source collaboration.

Rather than documenting every technical decision, this playbook explains **how engineering decisions should be made**.

Audience

This document is intended for:

- Core maintainers
- Backend engineers
- Frontend engineers
- DevOps engineers
- AI engineers
- UX engineers
- QA engineers
- Technical writers
- Open-source contributors

Every contributor should understand the principles described here before making significant architectural changes.

Relationship to Other Documents

The Engineering Playbook complements the other CivicOS documentation.

Document	Purpose
Vision Document	Why CivicOS exists

Technical Architecture	How CivicOS is designed
Experience Architecture	How CivicOS should feel
Product Roadmap	What gets built and when
Engineering Playbook	How software is built

Together, these documents form the complete engineering reference for CivicOS.

Engineering Philosophy

CivicOS is not built to demonstrate technical complexity.

It is built to create long-term public value.

Every engineering decision should therefore prioritize:

- Maintainability over cleverness.
- Simplicity over novelty.
- Reliability over speed of implementation.
- Transparency over hidden behavior.
- Modularity over monolithic design.
- Developer experience alongside user experience.

Technology should enable better civic outcomes rather than becoming an objective in itself.

Engineering Principles

Every contribution should align with the following principles.

---

1. Build for the Next Engineer

Code is read more often than it is written.

Every module, function, API, and service should be understandable by someone encountering it for the first time.

Readable software scales better than clever software.

---

2. Favor Simplicity

Whenever multiple solutions exist, prefer the simpler one unless there is a clear architectural reason not to.

Complexity should only be introduced when it creates measurable value.

---

3. Modular by Default

Every major capability should exist as an independent module or service.

New functionality should extend the platform rather than increase coupling between existing components.

A service should have one primary responsibility and communicate through well-defined interfaces.

---

4. APIs Are Products

Internal APIs deserve the same care as public APIs.

They should be:

- Consistent
- Well documented
- Versioned
- Predictable
- Backward compatible where practical

Poor internal APIs create long-term technical debt.

---

5. Data Has Clear Ownership

Each service owns its own data.

No service should directly modify another service's database.

Communication occurs through APIs or asynchronous events.

This preserves service independence and simplifies future scaling.

---

6. Security Is a Default

Security is not a final review step.

Every feature should consider:

- Authentication
- Authorization
- Validation
- Encryption
- Auditability
- Privacy

from its initial design.

---

7. Test Before Trust

Software should earn trust through automated verification.

Testing is part of development rather than a separate activity.

New functionality should include appropriate automated tests before it is considered complete.

---

8. Documentation Is Code

Architecture, APIs, workflows, and design decisions should be documented alongside implementation.

Undocumented systems eventually become difficult to maintain.

Documentation should evolve together with the codebase.

---

9. Optimize for Longevity

CivicOS is expected to evolve over many years.

Engineering decisions should consider:

- Future contributors
- Future deployments
- Future governments
- Future technologies

Short-term convenience should not compromise long-term sustainability.

Definition of Done

A feature is considered complete only when all of the following conditions are satisfied.

Functional

- Requirements implemented.
- Acceptance criteria satisfied.
- Edge cases considered.

Quality

- Code reviewed.
- Tests passing.
- Documentation updated.

Security

- Authorization verified.
- Validation implemented.
- Sensitive information protected.

Performance

- No obvious performance regressions.
- API latency acceptable.
- Database queries optimized.

Operations

- Logging implemented.
- Monitoring available.
- Deployment successful.

User Experience

- Error messages understandable.
- Accessibility considered.
- Mobile responsiveness verified where applicable.

Completing functionality alone does not mean a feature is complete.

Engineering Values

The CivicOS engineering team values:

Ownership

Engineers own problems rather than individual files.

Collaboration

Architectural decisions should emerge through discussion rather than individual preference.

Curiosity

Engineers should continually question existing assumptions and seek better solutions.

Humility

Good engineering welcomes constructive feedback.

No individual should become a bottleneck for progress.

Transparency

Design decisions should be explained rather than hidden.

Future contributors should understand not only *what* was built but *why*.

Public Impact

Every engineering decision ultimately affects citizens, communities, and public institutions.

Code quality therefore carries social responsibility.

Decision-Making Framework

When evaluating multiple technical solutions, contributors should ask:

1. Is it simpler?
2. Is it easier to maintain?
3. Does it reduce coupling?
4. Will another engineer understand it?
5. Can it scale without redesign?
6. Does it align with the architecture?
7. Does it improve the user experience?
8. Does it improve the developer experience?

If the answer to most of these questions is "yes," the solution is likely aligned with CivicOS engineering principles.

Closing Thoughts

Engineering is not simply the act of writing code.

It is the practice of building systems that remain understandable, reliable, and valuable over time.

The Engineering Playbook establishes the shared standards that allow CivicOS to grow from a single project into a global open-source platform without sacrificing quality or consistency.

Every contribution, regardless of size, becomes part of a larger mission to strengthen civic participation through technology.

Chapter 3

Development Standards

Overview

Development standards define how software is written throughout the CivicOS codebase.

Their purpose is not to restrict engineering creativity but to ensure that every contributor produces software that is consistent, maintainable, and easy to understand.

Regardless of experience or team, engineers should write code that feels as though it was produced by a single engineering organization.

Consistency is a feature.

General Engineering Principles

Every line of code should prioritize:

- Readability
- Simplicity
- Maintainability
- Predictability
- Testability
- Performance where appropriate

When trade-offs exist, readability should generally take precedence over cleverness.

Future contributors should understand code without needing its original author to explain it.

Programming Language Standards

For the initial implementation of CivicOS:

Layer	Technology
Frontend	React + TypeScript
Backend	NestJS + TypeScript
Database	PostgreSQL
Cache	Redis
Messaging	NATS (or Kafka in large deployments)
Infrastructure	Docker + Kubernetes
Package Manager	pnpm
Monorepo	Turborepo

Technology choices may evolve over time, but TypeScript should remain the primary language wherever practical to maximize code sharing and developer productivity.

TypeScript Standards

TypeScript should be treated as a tool for correctness rather than merely a replacement for JavaScript.

The project should enable strict compiler settings.

Avoid the use of:

any

Prefer:

unknown

or explicit types.

Every public function should define:

- parameter types
- return types
- exported interfaces

Type safety is part of CivicOS's reliability strategy.

Code Style

Formatting should be automated rather than manually enforced.

The project adopts:

- Prettier for formatting
- ESLint for linting

Contributors should never debate formatting in pull requests.

Automation should handle stylistic consistency.

Naming Conventions

Naming should communicate intent.

Variables

Use descriptive camelCase.

Good:

communityId

Poor:

id

Constants

Use UPPER_SNAKE_CASE only for true constants.

Example:

MAX_FILE_SIZE

Classes

PascalCase.

Example:

IssueService

Interfaces

Avoid unnecessary prefixes.

Good:

Issue

Prefer over:

IIssue

Enums

Use PascalCase.

Example:

IssueStatus

Enum values should remain readable.

Function Design

Functions should do one thing well.

Prefer:

validateIssue()

createIssue()

publishEvent()

Instead of:

createIssueAndNotifyEveryoneAndUpdateDashboard()

Large functions should usually be decomposed into smaller, reusable units.

File Size

Files should remain focused.

Recommended guideline:

- Target under 300 lines.
- Review files exceeding 500 lines.
- Consider refactoring files exceeding 800 lines.

Large files often indicate multiple responsibilities.

Class Design

Classes should follow the Single Responsibility Principle.

Each class should have one reason to change.

Example:

IssueService

should manage issue business logic.

It should not:

- send emails
- publish analytics
- manage authentication
- generate PDFs

Those responsibilities belong elsewhere.

Dependency Injection

Dependency Injection should be used consistently throughout backend services.

Dependencies should be injected rather than instantiated directly.

Good:

```
constructor(
  private readonly repository: IssueRepository
)
```

Avoid:

```
const repository = new IssueRepository();
```

Dependency Injection improves testing and flexibility.

Error Handling

Errors should be explicit.

Never silently ignore failures.

Every service should return predictable error structures.

Example:

```
{
 "code": "ISSUE_NOT_FOUND",
 "message": "The requested issue does not exist.",
 "requestId": "..."
}
```

Internal implementation details should never be exposed to API consumers.

Logging Standards

Logs exist for operators, not developers.

Every log entry should provide useful operational context.

Include:

- request ID
- service name
- user ID (when appropriate)
- organization ID
- execution time

Avoid logging:

- passwords
- access tokens
- secrets
- personal data unless absolutely necessary

Logs should support troubleshooting while respecting privacy.

Validation

Never trust client input.

Validation should occur:

- at API boundaries
- before database writes
- before external API calls
- before event publication

Validation failures should return clear, actionable messages.

Configuration Management

Configuration should never be hardcoded.

Use environment variables for deployment-specific settings.

Examples include:

- database URLs
- API keys
- AI provider credentials
- JWT secrets
- storage endpoints

Configuration should be centralized and validated during application startup.

Environment Variables

Environment variables should follow consistent naming.

Example:

DATABASE_URL

JWT_SECRET

REDIS_URL

NATS_URL

OPENAI_API_KEY

Variable names should be descriptive and consistent across services.

Comments

Good code minimizes the need for comments.

Comments should explain:

Why something exists.

Not:

What the code already says.

Good:

// Prevent duplicate issue creation during retry requests.

Poor:

// Increment counter.
counter++;

If code requires extensive explanation, consider simplifying the implementation.

TODOs

Temporary work should be tracked consistently.

Example:

TODO(civicos-421):
Replace temporary implementation once Notification Service supports batching.

Avoid anonymous TODOs that become permanent.

Feature Flags

New functionality that is incomplete or experimental should be protected using feature flags rather than long-lived branches.

Feature flags enable safer deployments and gradual rollouts.

Time Handling

Store all timestamps in UTC.

Convert to local time only when presenting information to users.

Avoid timezone-specific business logic unless explicitly required.

Identifiers

Use UUIDs as primary identifiers across services.

Avoid exposing sequential database IDs through public APIs.

UUIDs improve security and simplify distributed systems.

Asynchronous Programming

Prefer asynchronous operations where appropriate.

Long-running work should be delegated to background jobs rather than blocking user requests.

Examples include:

- notifications
- AI processing
- document generation
- search indexing
- analytics

Responsive systems improve user experience and scalability.

Code Review Checklist

Before requesting review, contributors should confirm:

- Code is readable.
- Naming is clear.
- Tests pass.
- Documentation updated.
- Logging added.
- Validation implemented.
- Security considered.
- No unnecessary complexity introduced.

Code review is a quality improvement process, not merely an approval step.

Closing Thoughts

Development standards establish a shared engineering language.

By consistently applying these conventions, CivicOS becomes easier to understand, easier to extend, and easier to maintain regardless of how many contributors join the project over time.

Strong engineering standards reduce friction, accelerate onboarding, and allow teams to focus on solving civic challenges rather than navigating inconsistent code.

What I would do today

Goal

By the end of today, you should have:

civicos/

apps/
  web/

services/
  api-gateway/
  identity-service/
  community-service/

packages/
  ui/
  types/
  config/

infrastructure/

docs/

running locally inside a monorepo.

No business logic.

No features.

Just the engineering foundation.

Step 1: Create the Repository

I would create:

mkdir civicos
cd civicos

git init

Then create a GitHub repository:

civicos

Public or Private?

For CivicOS:

Public.

Why?

- attracts contributors
- attracts grant reviewers
- shows progress
- builds credibility
- supports your future vision

You can always move sensitive work elsewhere.

Step 2: Create Turborepo

I would use Turborepo.

pnpm dlx create-turbo@latest .

Choose:

TypeScript
pnpm

This immediately gives:

apps/
packages/

Step 3: Clean Up Turbo

Remove demo applications.

Remove:

apps/docs
apps/web

We'll recreate them properly.

Step 4: Create CivicOS Structure

Create:

mkdir -p apps/web

mkdir -p services/api-gateway
mkdir -p services/identity-service
mkdir -p services/community-service

mkdir -p packages/ui
mkdir -p packages/types
mkdir -p packages/config

mkdir -p infrastructure

mkdir -p docs

Result:

civicos/

apps/
services/
packages/
infrastructure/
docs/

Exactly what we defined in the playbook.

Step 5: Create Web App

I recommend:

cd apps

```
pnpm create vite web \
  --template react-ts
```

Then:

pnpm install

Step 6: Create Identity Service

Inside:

services/identity-service

Create NestJS:

pnpm dlx @nestjs/cli new identity-service

Choose:

pnpm

Move generated files into:

services/identity-service

Step 7: Create Community Service

Repeat:

pnpm dlx @nestjs/cli new community-service

Step 8: Create API Gateway

Repeat:

pnpm dlx @nestjs/cli new api-gateway

Step 9: Create Shared Types Package

packages/types

Example:

export type UUID = string;

Later:

export interface User {}

export interface Community {}

export interface Organization {}

Step 10: Create Shared UI Package

packages/ui

This becomes:

Button
Input
Card
Modal
Badge
Table

used across:

web
admin
future apps

Step 11: Docker Foundation

Create:

docker-compose.yml

Start with:

```
postgres:
 image: postgres:16

redis:
 image: redis:7

nats:
 image: nats
```

Nothing more.

Keep it simple.

Step 12: First Commit

```
git add .

git commit -m "chore: initialize CivicOS monorepo"
```

What NOT to do today

Don't build:

- authentication
- communities
- organizations
- AI
- notifications
- RBAC

Not yet.

Today's objective is:

> Make the foundation exist.

That's all.

What I would do tomorrow

After the scaffold exists:

Sprint 0

Set up:

- ESLint
- Prettier
- Husky
- Commitlint
- Docker
- GitHub Actions

Then:

Sprint 1

Build only:

Identity Service

with:

POST /auth/register
POST /auth/login
POST /auth/refresh
GET /me

No more.

So what's next?

Sprint 0 – Engineering Foundation

This is where we become engineers again.

We'll work together on:

Repository setup

- Turborepo
- pnpm workspaces
- Folder structure

Development tooling

- ESLint
- Prettier
- Husky
- lint-staged
- Commitlint

Docker

- PostgreSQL
- Redis
- NATS

Shared packages

- `@civicos/types`
- `@civicos/config`
- `@civicos/ui`

Frontend

- React
- Vite

- Tailwind CSS
- TanStack Query
- React Router

Backend

- NestJS
- Prisma
- PostgreSQL

CI/CD

- GitHub Actions
- Test workflow
- Lint workflow

First service

Identity Service

My proposal for today

Forget documents.

Let's build.

I suggest we make today's goal:

> **"By the end of today, CivicOS compiles successfully as a monorepo."**

No authentication.

No database schema.

No APIs.

Just a clean, professional engineering foundation.

Once that's done, we'll commit it to GitHub as the first real version of CivicOS.